Here you added one more completed task, and you're using the ability to pass a completed date to differentiate the two. Note that the total size is now 10 instead of 7 and the total-size test needs to be changed accordingly.

Now the calculations you need for determining the projected project status are straightforward math based on this data:

basics/07/spec/models/project_spec.rb

```
Line 1  it "knows its velocity" do
        expect(project.completed_velocity).to eq(3)
        end
        
5       it "knows its rate" do
        expect(project.current_rate).to eq(1.0 / 7)
        end
        
        it "knows its projected days remaining" do
10      expect(project.projected_days_remaining).to eq(35)
        end
        
        it "knows if it is not on schedule" do
        project.due_date = 1.week.from_now
15      expect(project).not_to be_on_schedule
        end
        
        it "knows if it is on schedule" do
        project.due_date = 6.months.from_now
20      expect(project).to be_on_schedule
        end
```

You can quibble with some style choices in these tests. Even though the tests are against the Project class, they have a stealth dependency on the Task class also working. That's not ideal, as it makes it harder to determine the cause if the test fails. In Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, I'll go over some strategies for breaking this kind of dependency in tests.

I also use a couple of different strategies for dealing with math. The assertion on line 6 has a mathematical answer expressed as a math expression in the test (1.0 / 7), while the assertion on line 10 does all the math and spits out the final answer (35). The algebraic version is more clear because it describes the way the answer is derived (and, in this case, makes it easier to express a floating-point answer), whereas the numerical version can seem magical—why 35? However, the downside of having code expressions in test assertions is that it encourages using the test to directly describe the final code, by copying and pasting the code from the test. It's usually better to have the implementation code be as independent as possible from the test itself. Also, when comparing floating-point numbers as you are in this case, the implementation